

Interview Case: CRM System Development with .NET Core

Objective

Create a basic CRM system that allows managing customer data. The system should be built using **.NET Core** for the backend, **PostgreSQL** as the database, and either **React.js** or **Razor Pages** for the front end. The system should include basic CRUD operations for customer management, user (login/logout), and the ability to filter customers by various criteria. The project should demonstrate a good use of C# collections (like arrays, lists) for efficient data handling.

Requirements

1. Backend (.NET Core)

- **Language:** C#
- **Framework:** .NET Core
- **Database:** PostgreSQL
- **Functionality:**
 - **CRUD Operations:** Implement Create, Read, Update, and Delete operations for customer data.
 - **User Authentication and Authorization:**
 - Implement authentication using JWT (JSON Web Tokens) or Identity framework for login/logout functionality.
 - Ensure role-based access control for different users.
 - **API Endpoints:**
 - Create RESTful API endpoints to filter customers based on criteria (e.g., name, email, registration date, region).
 - **Error Handling:** Proper error handling and validation for input data.
 - **Logging:** Log important actions like login attempts, CRUD operations, and filtering actions using **ILogger**.

2. Database (PostgreSQL)

- **Entities:**
 - **User:**
 - **Fields:** **Id**, **Username**, **Password**, **Role**, **CreatedAt**, **UpdatedAt**
 - Used for authentication and role-based access.
 - **Customer:**
 - **Fields:** **Id**, **FirstName**, **LastName**, **Email**, **Region**, **RegistrationDate**
 - This table stores customer data that is managed by the system.

3. Frontend (React.js or Razor Pages)

- **Technology:** Either React.js or .NET Core with Razor Pages.

- **Functionality:**
 - **Login Page:** Users should be able to log in and access the CRM dashboard.
 - **Customer Management Page:**
 - List customers in a table with search and filtering options.
 - Form to add a new customer.
 - Edit and delete options for each customer.
 - **Filter Options:** Allow users to filter customers by name, region, registration date, etc.
 - **Responsive Design:** Ensure the UI is responsive for both mobile and desktop.

4. Additional Requirements

- **RESTful API:** Expose CRUD operations and filtering functionalities via REST API endpoints.
 - **Code Quality:** Follow best practices for code structure, naming conventions, and documentation.
-

Mock Data

Provide some mock customer data to use in the system. This data should include customers with various names, emails, regions, and registration dates.

```
INSERT INTO customer (FirstName, LastName, Email, Region,
RegistrationDate) VALUES
('John', 'Doe', 'john.doe@example.com', 'North America', '2023-06-
15'),
('Jane', 'Smith', 'jane.smith@example.com', 'Europe', '2023-05-10'),
('Carlos', 'Gomez', 'carlos.gomez@example.com', 'South America',
'2023-07-22');
```

Evaluation Criteria

- **Backend Implementation:** Correct and efficient use of .NET Core, C# collections, LINQ, and PostgreSQL integration.
- **Frontend Design and Functionality:** Clean, user-friendly design with working CRUD operations and filtering.
- **Security:** Proper implementation of user authentication, authorization, and access control.
- **Code Quality:** Well-structured, maintainable, and efficient code with appropriate use of design patterns (e.g., Repository Pattern).

GitHub Repository

- **Branching Strategy:**
 - **main:** This branch contains the final, stable version of the project.
 - **development:** This branch contains the active development work.
- **Commit Structure:**
 - Commits should be frequent and reflect logical steps in development (e.g., "Add customer entity", "Implement login functionality").
 - Commit messages should be descriptive and explain what was changed and why.
 - Code comments should be used to explain complex logic.
- **Pull Request and Merging:**
 - Once development work is complete and tested on the development branch, create a pull request to merge changes into the main branch.
 - Provide a clear explanation of what has been implemented and why during the pull request.

Note: Please keep your GitHub account private, as you will be asked to share it with us during the process.